

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

2026-09-22

Rules, judge, loop

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Timing, dividers included: open + check-in 5 · git 7 · from a model to an agent 18 · delegation workflow 12 · live kick-off 10 · your turn 30 · retro 4 · wrap 1 = 87, with three minutes of slack. Per-slide weights are in the plan, not on the slides. If behind at the agent divider, cut the history slides (12–14, NVIDIA included), and the two GPT-2 slides (15–16) are the next cut. TA-run. Slides carry ideas only: every command is on the Week 4 page, open on their laptops, and in these notes. Presenting laptop: SplitIt with feature 1 merged, clean main, Codex open, GitHub Desktop signed in, data reset.

Last week, this week.

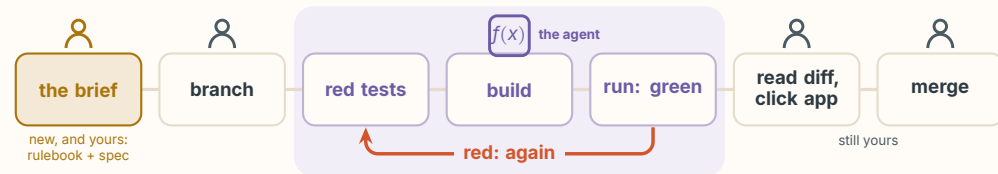
LAST WEEK

you, at every station



TODAY

same stations; three of them run without you



Same stations. Three now run without you. You write the brief instead.

2026-09-22

Rules, judge, loop

└ Last week, this week.

Last week, this week.



One minute, before the agenda. Top row: what they did on stage last week, branch, red tests, paste the spec, read the diff, green, click, merge, and every step had their hands on it. Bottom row: today the four in the middle run inside one Codex turn without them, red to green, round and round; the only new thing is the first chip. The brief is two files, the rulebook for every task and the spec for this one, and the class is about those two files and about how to check the purple part without watching it. Everything until Act 3 is what the purple part is made of. Then the agenda.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Delegating: the brief, the judge, the loop
- 5 Build one, live — then you build one
- 6 Retro: one new rule

2026-09-22

Rules, judge, loop

└ Today.

Three pictures carry the first half, and each one is built up over several slides rather than shown at once: the git graph, the model-to-agent stack, and the ReAct loop. Say so: "you will see the same picture grow." That is the whole visual method of the class.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Delegating: the brief, the judge, the loop
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

2026-09-22

Rules, judge, loop

└─ Your GitHub page.

Walk the room looking at browsers, not code. Thirty seconds a row; note who is behind, move on. They can pull main and run the feature-1 tests while you walk; the command is on the Week 4 page. Don't debug git at the front: the next six slides are the explanation, and the page has the "when git is confusing" table from last week.

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

Why git exists.



"I'm an egotistical bastard, and I name all my projects after myself. First Linux, now git."

- 2005. The Linux kernel: thousands of people mailing changes to one man — and the commercial tool that tracked them revoked its free licence.
- He wrote git himself, in about ten days, and was running the kernel on it two months later.
- Everyone holds the **whole history**. Every change is named by its contents, so nothing can be altered quietly. And it is **fast**.

2026-09-22

Rules, judge, loop

└─ Why git exists.

A minute, and it replaces the divider: the act opens on a person. The story, in the order of the cards. 2005: Linus Torvalds has run the Linux kernel for fourteen years, thousands of contributors, changes arriving by email. Since 2002 he had tracked them with BitKeeper, a commercial tool that let the kernel use it free; in April 2005 that permission was withdrawn after another kernel developer reverse-engineered its protocol, and the kernel had nothing. Torvalds looked at the alternatives, decided they were all too slow ("take CVS as an example of what not to do"), and wrote his own: first commit 7 April 2005, the kernel's own history moved into it by June, the 2.6.12 release managed in git that month. Then the three design decisions, which are the three ideas of the next eight slides: every developer has the entire history on their own machine, not a copy of the current files — that is the local/remote slide; every commit is named by a hash of its contents, so history cannot be rewritten without everyone noticing — that is the record slide; and it had to merge and branch in seconds, because the kernel did it hundreds of times a day. He handed maintenance to Junio Hamano within months and has been a user since. The name: git is British slang for an unpleasant person; the quote is his own joke about it. Then turn: here is how a change gets into main. If anyone asks about GitHub: a company, 2008 that hosts git repositories and added the pull request: not the same thing.

LINUS TORVALDS · APRIL 2005

Why git exists.



"I'm an egotistical bastard, and I name all my projects after myself. First Linux, now git."

- 2005. The Linux kernel: thousands of people mailing changes to one man — and the commercial tool that tracked them revoked its free licence.
- He wrote git himself, in about ten days, and was running the kernel on it two months later.
- Everyone holds the **whole history**. Every change is named by its contents, so nothing can be altered quietly. And it is **fast**.

main



One version, one history, one truth. No folder full of *model_v3_FINAL_jm*.

```
$ git show
```

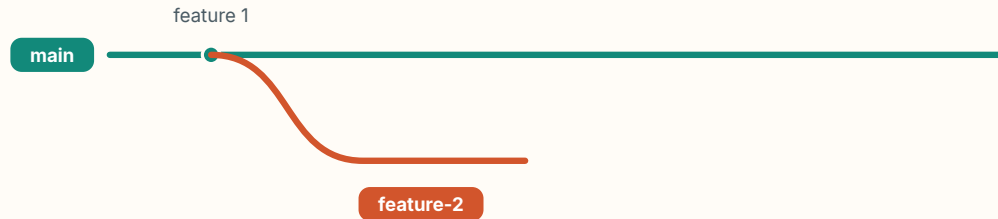
```
diff --git a/logic.py b/logic.py
+def balances(group_id):
+    return paid(group_id) - shares(group_id)
```

2026-09-22 Rules, judge, loop

main



A branch.



A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

```
$ git diff main feature-2
```

no output — a new branch is an identical copy

2026-09-22

Rules, judge, loop

└─ A branch.

A branch.

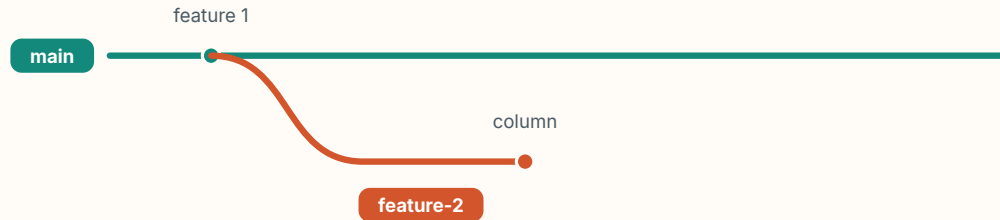


A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

\$ git diff main feature-2

no output — a new branch is an identical copy

A commit.



A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

```
$ git show
```

```
diff --git a/db.py b/db.py
-      "date"]
+      "date", "category"]
```

2026-09-22

Rules, judge, loop

└─ A commit.

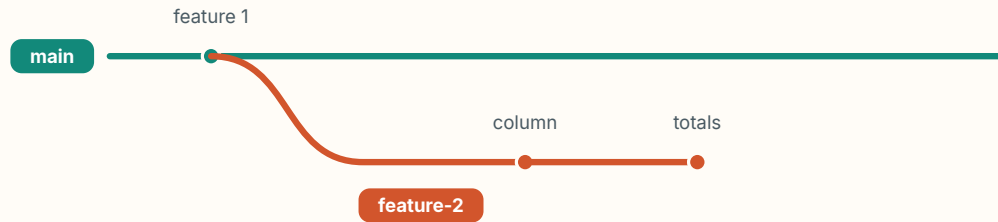
A commit.



A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

```
$ git show
diff --git a/db.py b/db.py
columns = ["id", "payer", "amount",
+         "date", "category"]
```

Working on a branch.



Two commits in, the tests are still red, the feature is half-built. Everyone else's `main` is untouched.

```
$ git diff main feature-2
```

```
diff --git a/db.py b/db.py
-      "date"]
+      "date", "category"]
diff --git a/logic.py b/logic.py
+def category_totals(group_id):
+    return sums_by("category", group_id)
```

2026-09-22

Rules, judge, loop

Working on a branch.

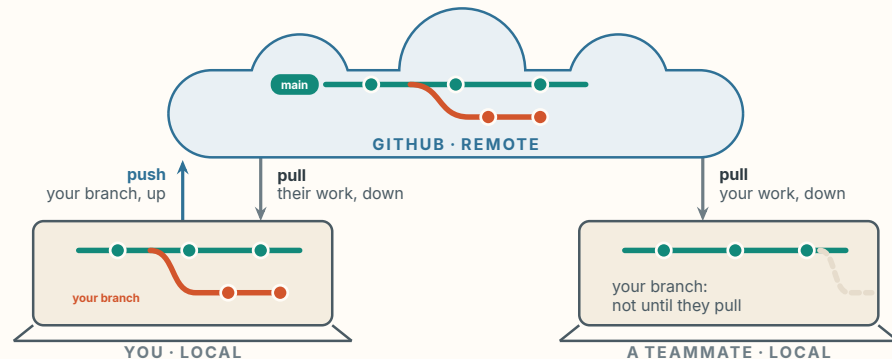
Working on a branch.



Two commits in, the tests are still red, the feature is half-built. Everyone else's `main` is untouched.

```
$ git diff main feature-2
diff --git a/db.py b/db.py
+      "date"]
+      "date", "category"]
diff --git a/logic.py b/logic.py
+def category_totals(group_id):
+    return sums_by("category", group_id)
```

Local, and remote.



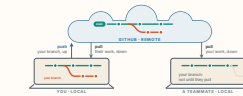
One copy in the cloud, one on each laptop, and nothing crosses by itself. **Push** sends yours up, **pull** brings theirs down. A **pull request** asks someone to read it before **main** takes it.

2026-09-22

Rules, judge, loop

Local, and remote.

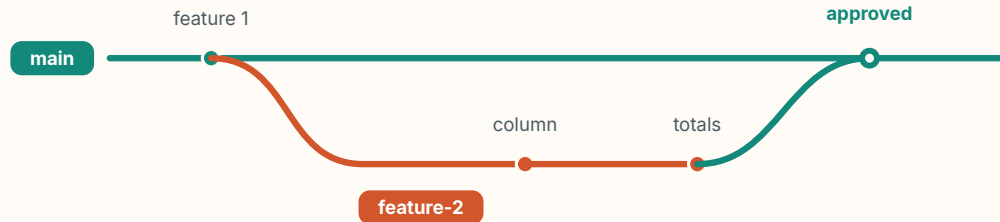
Local, and remote.



One copy in the cloud, one on each laptop, and nothing crosses by itself. **Push** sends yours up, **pull** brings theirs down. A **pull request** asks someone to read it before **main** takes it.

The only slide in the act that leaves the graph, and the one that fixes the vocabulary: *local* is the copy on a laptop — yours, or anyone's — and *remote* is the one copy on GitHub that all of them talk to. Walk it: your branch is on your screen. Push, and it is in the cloud too. Look at the teammate's screen: not there — and it will not be until they pull. That is the whole meaning of "remote": one shared copy, and everything else is a laptop that has to ask. The three moves are the only things that cross the gap: push up, pull down, and a pull request, which is asking someone to read the diff before the cloud's main takes it. If someone asks where "add"/"staging" went: GitHub Desktop turned it into the checkboxes next to each changed file, so they have already used it without the name.

Merge.



Merge is the sign-off. `main` now has your work, and the next person starts from there.

```
$ git diff main~1 main
```

```
diff --git a/db.py b/db.py
-      "date"]
+      "date", "category"]
diff --git a/logic.py b/logic.py
+def category_totals(group_id):
+    return sums_by("category", group_id)
```

2026-09-22

Rules, judge, loop

└─ Merge.

Merge.

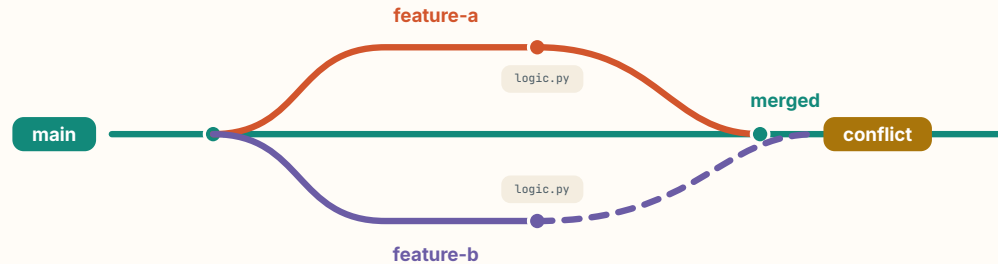


Merge is the sign-off. `main` now has your work, and the next person starts from there.

```
$ git diff main~1 main
diff --git a/db.py b/db.py
+      "date", "category"]
diff --git a/logic.py b/logic.py
+def category_totals(group_id):
+    return sums_by("category", group_id)
```

Land the cycle: they are back at slide one, with a bigger main. That is the habit. Two rules fall out of the story rather than being announced: name the branch after the job (slide 2), and keep commits small enough to explain (slide 3). The diff is the same six lines as two slides ago, asked of main instead of the branch: the column and the totals are main's now. If someone asks what happens to the branch: it is finished, GitHub offers to delete it, say yes. Then turn: "the thing that will make those commits today is not you. So: what is it?"

A conflict.



Different files, git merges by itself.
When two branches change **the same lines**, it stops and asks which version is right.

```
$ cat logic.py
```

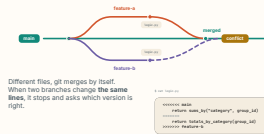
```
<<<<<< main
    return sums_by("category", group_id)
=====
    return totals_by_category(group_id)
>>>>>> feature-b
```

2026-09-22

Rules, judge, loop

└─ A conflict.

A conflict.



Different files, git merges by itself.
When two branches change the **same lines**, it stops and asks which version is right.

```
<<<<<< main
    return sums_by("category", group_id)
=====
    return totals_by_category(group_id)
>>>>>> feature-b
```

Forty seconds, and say up front that it is an aside: nobody hits this today, because each of them is alone on their own repo. It is here so the word is not new in Block 3, when they share one. Point at the fork — both branches start from the same commit, and both change the same line of logic.py. Feature-a comes home first, so main moves. Then feature-b arrives with an edit to a file that no longer looks the way it did when it left, and git stops. Be precise about when: it merges changes to different files on its own, and it merges changes to different parts of the same file on its own; it refuses only when the same lines changed on both sides. The fix is not a git command, it is a person reading both versions and choosing. Say the reassuring half out loud, because the word frightens people: a conflict is not a mistake, nothing is lost, and it is git declining to guess. How you actually resolve one is Block 3 — GitHub Desktop shows both versions side by side and you pick. What prevents most of them is already on the slides behind you: small commits, short-lived branches, and pull before you start.

The record.

HISTORY

Merge feature-2: categories

you · today, 10:42

c41d8e2

Add the totals

you · yesterday, 17:05

7d21e0b

Add the category column

you · yesterday, 14:02

4c8ab77

Feature 1: balances

you · 15 Sep

91e04d5

4c8ab77

Add the category column · you

go back to here

db.py

COLUMNS = ["id", "payer", "amount",

- "date"]

+ "date", "category"]

1 file changed · 1 line

Every commit on record: **what** changed, **who** did it, **when**. Open any one and read its diff — or go back to it.

2026-09-22

Rules, judge, loop

The record.

Forty seconds. This is the graph from the last six slides turned on its side, newest at the top: the merge, the branch's two commits in its colour, feature 1. Read one row across: a hash, which is the commit's name; the message; who; when. Then the two things it buys. First, the audit: pick any row and you get its full diff — the highlighted one is the column commit from five slides ago, and that is its diff beside it. Today's commits will be theirs, of the agent's work, and this is how they check what went in after the fact, not only at the moment they clicked accept. Second, the chip: any row can be gone back to, and nothing above it is deleted by doing so; the way to do it is in GitHub Desktop, and it is Block 3. The panel is GitHub Desktop's History tab, drawn: click a row on the left, its diff appears on the right. If someone asks about the hash: seven characters is enough to name a commit; the full one is forty.

The record.

HISTORY

Merge feature-2: categories

you · today, 10:42

c41d8e2

Add the totals

you · yesterday, 17:05

7d21e0b

Add the category column

you · yesterday, 14:02

4c8ab77

Feature 1: balances

you · 15 Sep

91e04d5

4c8ab77

Add the category column · you

go back to here

db.py

COLUMNS = ["id", "payer", "amount",

- "date"]

+ "date", "category"]

1 file changed · 1 line

Every commit on record: **what** changed, **who** did it, **when**. Open any one and read its diff — or go back to it.

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

2026-09-22

Rules, judge, loop

agent

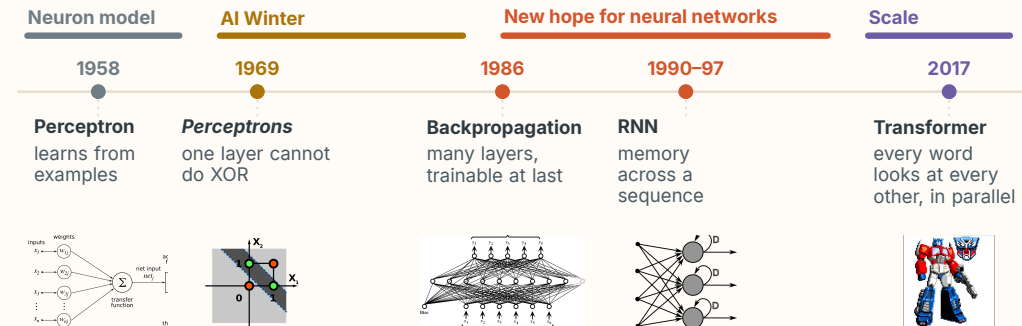
FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

Sixteen minutes. Two history slides first, so the thing feels like engineering rather than magic; then the stack is built in four moves; then one tool call in full; then reasoning, watched; then the loop gets its name; then what it can hold. Every slide in the act is one picture and one sentence. If you are behind, the two history slides are the ones to cut.

AI research chronology.



The ideas were all there. What research still needed was **compute** and **data**, and both arrived after 2007.

Rosenblatt (1958) · Minsky & Papert (1969) · Rumelhart, Hinton & Williams (1986) · Elman (1990) · Hochreiter & Schmidhuber (1997) · Vaswani et al. (2017)

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (public crawls from 2011)

2026-09-22

Rules, judge, loop

AI research chronology.

AI research chronology.



Ninety seconds, left to right. The dates are on the slide; the names and the story are here. 1958, Rosenblatt's perceptron: a machine that adjusts its own weights from examples rather than being programmed rule by rule. Still the whole idea. 1969, Minsky and Papert's book *Perceptrons*: one layer can only separate what a straight line separates, so it cannot learn XOR. Funding collapsed; the first AI winter. Careful with the attribution: they wrote the critique that made multi-layer nets the open problem, not the nets themselves. 1986, Rumelhart, Hinton and Williams in *Nature*: backpropagation sends the error back through every layer, so many layers finally become trainable. The answer to 1969, seventeen years late. Recurrence, two papers: Elman (1990) gives a network memory of what came before; LSTM (1997) makes it survive a long sequence. 2017, Vaswani et al.: attention drops the chain — every word looks straight at every other, which is why long text holds together and why it trains in parallel. Then the caption, which is the payoff: none of this is new. What arrived was compute — CUDA in 2007 let ordinary programmers run their maths on a graphics card, and in 2012 AlexNet trained on two of them and cut the ImageNet error by nearly half — and data: Common Crawl, a non-profit giving the public web away as text. Neither is an idea about intelligence. If someone asks why now and not in 1990, that is the answer.

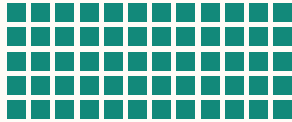
Gaming brought cheap parallel arithmetic.

A CPU



a few, fast · one thing each

A GPU



thousands, slower · all at once

CUDA · 2007

Any maths, by any programmer.



MINING · 2015+

Billions of hashes at once.



ALEXNET · 2012

Two gaming cards win images.



BERT · 2018

Language, on eight GPUs.



LLMs · 2020 →

Tens of thousands of GPUs.

NVIDIA made GPUs for gamers, but CUDA opened them for general-purpose computing. Training a neural network is nothing but parallel matrix arithmetic.

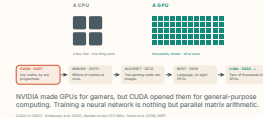
CUDA 1.0 (2007) · Krizhevsky et al. (2012), AlexNet on two GTX 580s · Devlin et al. (2018), BERT

2026-09-22

Rules, judge, loop

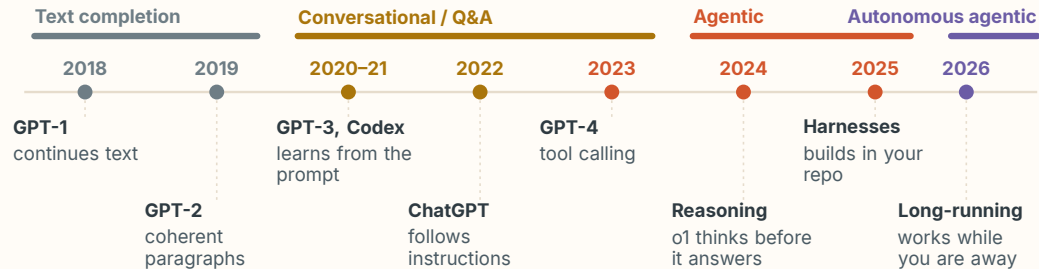
└ Gaming brought cheap parallel arithmetic.

Gaming brought cheap parallel arithmetic.



A minute; it is an aside, but it is the one company name in the room everyone knows, and the answer is a software story, which is the run-up slide's point. Ask it aloud, it is not on the slide: *why is NVIDIA the most valuable company in the world?* Take a guess or two (chips, AI). Then the answer: CUDA, 2007. A graphics card draws millions of pixels at once, and no pixel waits for another, so the chip is thousands of small slow workers instead of a few fast ones. That is the picture at the top. Until 2007 you could only use it by pretending your problem was a picture. CUDA let any programmer write ordinary maths for it. Then walk the strip, left to right: the same trick, bigger each time. Mining, billions of hashes side by side, and the 2017 and 2021 card shortages. AlexNet in 2012 on two gaming cards, cutting the ImageNet error by nearly half. BERT in 2018, when the trick jumped from images to language. Now one model on tens of thousands of chips at once, because training is nothing but multiplying big grids of numbers. Caveat if someone looks it up: Google trained BERT on its own TPUs; eight GPUs is the box people quote for reproducing it. A milestone in the trick, not a sale. The valuation: NVIDIA passed four trillion dollars in July 2025, the first company to; say "as of this year" and check the number the week you teach. The lesson to carry: the ideas were old, the compute arrived, and the thing that unlocked the compute was a piece of software.

2018 onwards.



2026-09-22

Rules, judge, loop

└─ 2018 onwards.

2018 onwards.



Two minutes, one sweep left to right. Say the four band names as you go (text completion, conversational, agentic, autonomous agentic). That is the shape to remember, not the model names. Then say the hand-off out loud, since it is no longer written on the slide: the job changed twice — when it learned to take instructions (2022) and when it got tools and a loop (2023 onwards) — and the next four slides build exactly that second change. Dates are approximate release months on purpose. If asked, there is a size story behind these dates too: parameters went from 117 million (GPT-1) to 1.5 billion (GPT-2) to 175 billion (GPT-3) in two years, then stopped being published. Size stopped being the whole story around 2022, and what changed after that is on the right of the timeline.

GPT-2 writes a paragraph.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

Radford et al. (2019), *Language Models are Unsupervised Multitask Learners*, table 13 · OpenAI, 14 February 2019

2026-09-22

Rules, judge, loop

└─ GPT-2 writes a paragraph.

One minute, and read it aloud; the room will laugh at the right place. This is the sample OpenAI published with GPT-2 in February 2019, the one that made the field sit up: a model that only predicts the next word, given three sentences, keeps a news story going with a named scientist, a university, and quotes. Two things to plant. First, "completing": nobody asked it a question, it continued the text, which is all a language model does and is the top-left band on the timeline. Second, the four horns: two sentences in, a unicorn has four of them, and the prose never flinches. Fluency and truth are different properties, and only one of them is what the model is trained for. That is why Act 3 has a judge. Say it was the best of ten tries; the paper says so.

GPT-2 writes a paragraph.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

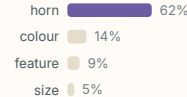
Radford et al. (2019), *Language Models are Unsupervised Multitask Learners*, table 13 · OpenAI, 14 February 2019

Guess the next word.

1 · HIDE A WORD

The scientist named the population,
after their distinctive _____

2 · GUESS



3 · CORRECT

the page says: horn

off by a bit?
get corrected

next word, next page · a few billion times

That is all training is: hide the next word, guess, get corrected.

Trained on 40 GB of text: 8 million pages Reddit users had linked to · Radford et al. (2019), *Language Models are Unsupervised Multitask Learners*

2026-09-22

Rules, judge, loop

Guess the next word.

Guess the next word.



A minute and a half; it is the only GPT-2 slide, so land both halves. Walk the strip left to right, then round the loop. One: a page from the web, next word covered. Two: the model guesses, a whole list, each with a probability. Three: the page is uncovered, horn; if the model put too little weight on horn, every one of its 1.5 billion numbers is nudged a hair towards it. Then the next word, the next page, for weeks. Nobody typed in a fact or a grammar rule. If asked about the data: not Common Crawl, which OpenAI called too messy, but WebText, 8 million pages that Reddit users had linked to, 40 GB; GPT-3 read a filtered Common Crawl a year later. The percentages are made up; the real list has 50,257 entries, one per token. Second half, the hand-off: writing is the same loop with no page to check against. Pick a word from the guess, stick it on the end, run again; every word of the unicorn story came out that way. No plan, so a fluent paragraph can walk into four horns and keep going: there is no step three this time, no page to uncover, so the guess is the output. That gap is what the rest of the class fills: tools that let it look, a judge that says red or green, and rules. The agent loop a few slides on is this loop one level up, with a tool call instead of a word.

A language model.



Text in, text out. It cannot open a file or run anything: you copy in, you paste out, and **you** are the one who finds out whether it works.

2026-09-22

Rules, judge, loop

└ A language model.

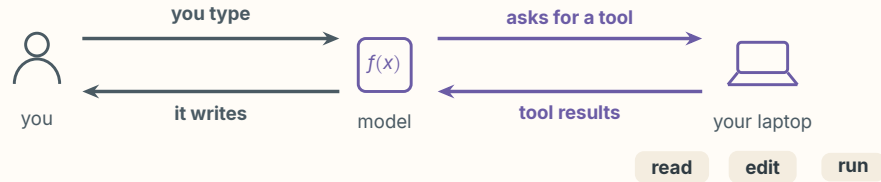
A language model.



Text in, text out. It cannot open a file or run anything: you copy in, you paste out, and **you** are the one who finds out whether it works.

This is ChatGPT in a browser, and it is what they used all through Class 2: it advised, they carried it out, and they were the only one who ever found out whether it worked. Point at the empty half of the slide and say it: everything else in this act goes there. Say "LLM" once, so they have the acronym: large language model. Then never again.

Tools.



A **tool** is an action the model can request but cannot carry out itself: read a file, edit a file, run a command. Tool results are shown to **the model**.

2026-09-22

Rules, judge, loop

Tools.

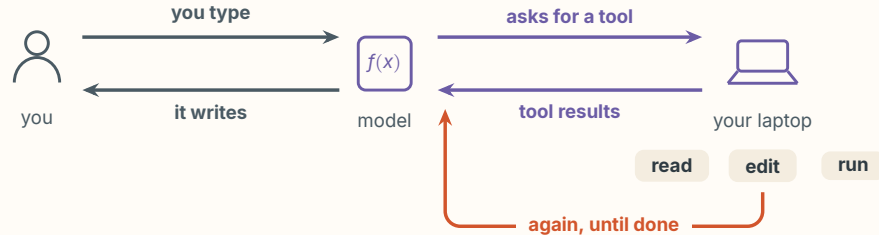
Tools.



A **tool** is an action the model can request but cannot carry out itself: read a file, edit a file, run a command. Tool results are shown to **the model**.

Point at the lower arrow and stop there: that is the slide. In Class 2 the result came back to you, and only when you got round to it. Here it comes back to the model, immediately. That is the only reason a loop is possible at all. The model still cannot open anything — it can only ask, which is what the next slide but one shows in full.

An agent.



Agent = model + tools + a loop. It acts, sees the result, and chooses again.

2026-09-22

Rules, judge, loop

└─ An agent.

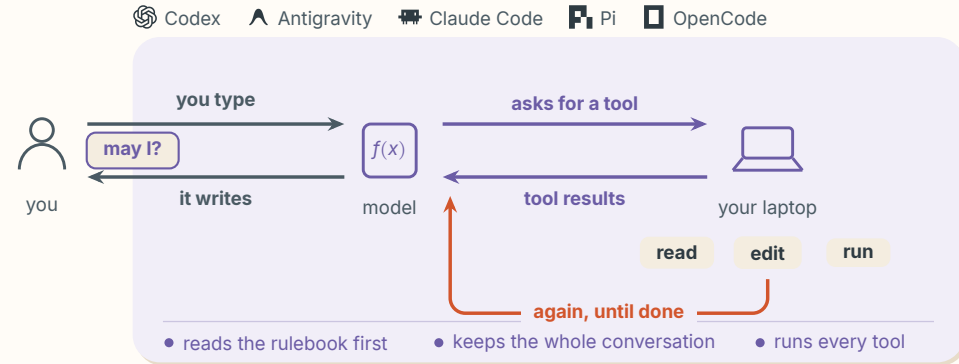
An agent.



Agent = model + tools + a loop. It acts, sees the result, and chooses again.

One new arrow, and it is the definition. Run a test, read the failure, edit, run again: that chain of tool calls is the loop. "Agentic" just means: can act, not only answer. Ask them what they saw Codex do last week and translate it: "that was a read, that was an edit, that was a run, and it kept going."

The harness.



A **harness** wraps a model with tools, instructions of its own and a loop tuned for coding — and it manages the context.

2026-09-22

Rules, judge, loop

└─ The harness.

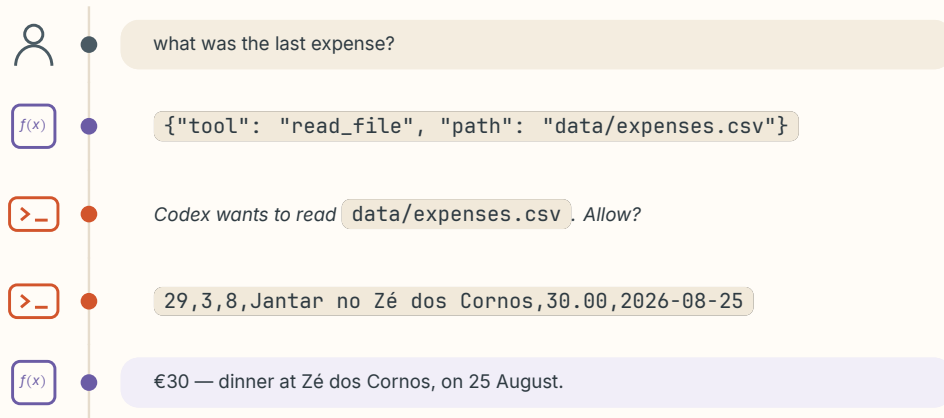
The harness.



A harness wraps a model with tools, instructions of its own and a loop tuned for coding — and it manages the context.

The box was always there; now it has a name. The model writes a tool call; the harness is what actually runs it, and it is the thing that stops to ask you — that approval prompt they click through all lesson is the harness at its own edge. It also loads AGENTS.md at the start and keeps the whole conversation, which are the two hooks the three standards hang on. Say it plainly: the standards are you configuring the harness. Harnesses differ in tools, permissions and how they manage the conversation; the loop is the same, which is why this class is not about Codex.

End-to-end flow.

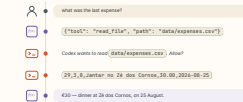


2026-09-22

Rules, judge, loop

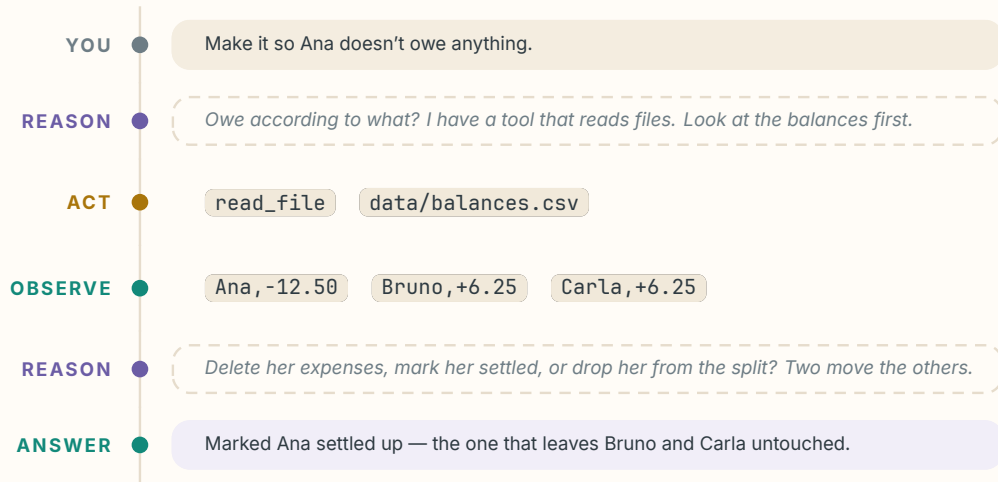
End-to-end flow.

End-to-end flow.



Read it top to bottom, slowly. This is the slide that turns the previous four from a diagram into a mechanism. It is the same spine as the reasoning slide that follows — one turn per box, the loop down the left edge — so the next slide reads as this one with thinking added rather than as a new picture. Down the spine is who is acting, as an icon: the person and the boxed $f(x)$ are the same two from the stack slides, and the window with a prompt in it is the harness — Codex on their laptops. If anyone asks about $f(x)$: that is the claim of the history act in one glyph — the model is a function, text in and numbers out, not a mind. One colour per actor, so the eye can count three of them and see that the harness does two of the five steps. Three things to point at. (1) The model's output is not prose: it is a small structured request naming a tool and what to use it on. It cannot open anything; it can only ask. (2) Codex — the harness — is what actually touches the disk, and it stops to ask you first. (3) The file's real line is pasted into the conversation as ordinary text, and from there the model reads it like any other message. Ask the room: "if we had said no, what could it have answered?" Nothing. The JSON is illustrative; each harness has its own wire format, and they never type it.

Reasoning.



Wei et al. (2022), *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, NeurIPS 2022 · arxiv.org/abs/2201.11903

2026-09-22

Rules, judge, loop

Reasoning.



Read the ask aloud and stop. Ask the room what it means — you will get at least two answers, which is the point. Then walk the spine. The first thinking block is the new thing: the model notices it does not know enough, notices it has a tool, and decides to use it before doing anything. That is a decision, made in words, before any action. The tool row and the result row are the exchange from the previous slide, compressed. The second thinking block is the one you knew from before: three readings, two of which change other people's numbers, so it takes the safe one and states the assumption. Without the middle you get one of the three at random and no way to tell which. What reasoning is: the model writes its working out before it answers, and the steps it writes are the extra thinking — a model spends a fixed effort per word, so working it out on the page is how it buys more. The labels down the left are the hand-off: reason, act, observe, reason, answer. Next slide gives that a name. The citation is where the technique got its name: Wei et al. 2022 found that asking a model to write its steps beat asking for the answer. Lineage if asked: scratchpads (Nye et al. 2021) came first, and OpenAI o1 (2024) was the first model trained to do it unprompted; DeepSeek-R1 and Claude's extended thinking followed. The honest part: a guess and a worked answer look equally confident on screen, which is why you still read the diff. Tie to standard 1: "state your assumption" is a rule you can put in AGENTS md

Prompt engineering, 2022–2024.

WHEN	PROMPT	EFFECT(S)	SOURCE
2022	<i>"Let's think step by step."</i>	maths score 18% → 79%, same model	Kojima et al., NeurIPS
2023	<i>"Take a deep breath and work on this problem step-by-step."</i>	maths score 72% → 80%, over "step by step"	Yang et al., Google DeepMind
2023	<i>"This is very important to my career."</i>	+8% on instruction tasks	Li et al.
2023	<i>"I'm going to tip \$200 for a better solution!"</i>	+45% "quality" on GPT-4, judged by hand on twenty questions	Bsharat et al., MBZUAI
2024	<i>"Please"</i> and <i>"thank you"</i>	rude prompts score lower; polite ones cost "tens of millions" in power	Yin et al. · Altman, on X (2025)

Since 2024: don't. Reasoning models work it out first. OpenAI's guide for them: *avoid chain-of-thought prompts; "think step by step" "can sometimes hinder"*.

OpenAI, Reasoning best practices

2026-09-22

Rules, judge, loop

└ Prompt engineering, 2022–2024.

One minute, and it is a laugh slide with a point. Read two or three rows aloud: a model told to take a deep breath got eight points more on school maths than one told to think step by step, and nobody wrote that sentence: another model found it by trying prompts (say this, it is not on the slide). Offer a tip and the judged quality went up by nearly half, on twenty questions, so hold the number loosely. Same for the career line: a 2024 replication (arXiv 2409.20303) found about 1slide shows the claim, you carry the caveat. The room will have done some of this. Then the honest part: these were real effects on real models, because the model is the next-word machine from the GPT-2 slides, and the words that sat next to careful answers on the web were words like "let's think step by step". The purple box is the point: since o1 in 2024 the model writes its working out on its own, and OpenAI's own guide for those models says not to ask for it, it can hurt. That is the reasoning slide before this one. What survives of prompt engineering is the next act: say what the task is, what not to touch, what done looks like, and what to show you. Structure, not incantations. If someone asks about please and thank you: it does not hurt, it costs electricity, and it is a habit worth keeping for the humans reading your messages.

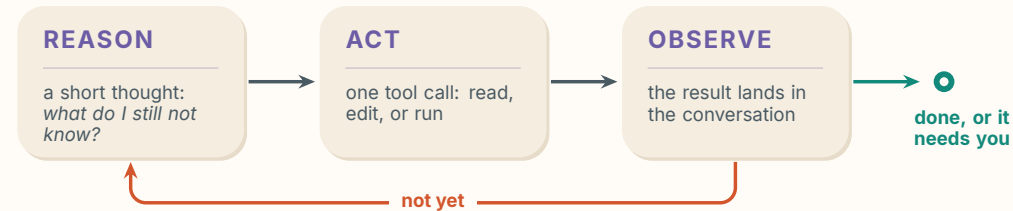
Prompt engineering, 2022–2024.

WHEN	PROMPT	EFFECT(S)	SOURCE
2022	<i>"Let's think step by step."</i>	maths score 18% → 79%, same model	Kojima et al., NeurIPS
2023	<i>"Take a deep breath and work on this problem step-by-step."</i>	maths score 72% → 80%, over "step by step"	Yang et al., Google DeepMind
2023	<i>"This is very important to my career."</i>	+8% on instruction tasks	Li et al.
2023	<i>"I'm going to tip \$200 for a better solution!"</i>	+45% "quality" on GPT-4, judged by hand on twenty questions	Bsharat et al., MBZUAI
2024	<i>"Please"</i> and <i>"thank you"</i>	rude prompts score lower; polite ones cost "tens of millions" in power	Yin et al. · Altman, on X (2025)

Since 2024: don't. Reasoning models work it out first. OpenAI's guide for them: *avoid chain-of-thought prompts; "think step by step" "can sometimes hinder"*.

OpenAI, Reasoning best practices

The ReAct loop: the standard agent architecture.



Reason, act, observe, repeat. Every harness today *still* uses this simple pattern.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

2026-09-22

Rules, judge, loop

└─ The ReAct loop: the standard agent architecture.

The ReAct loop: the standard agent architecture.

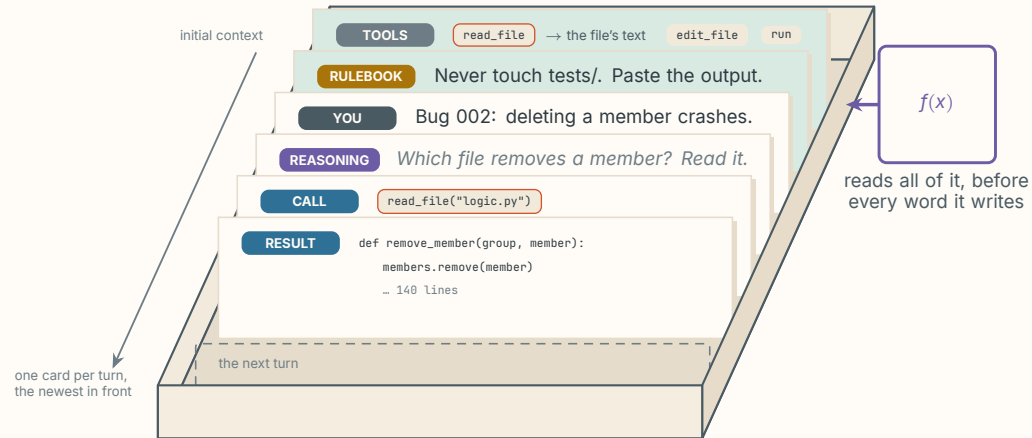


Reason, act, observe, repeat. Every harness today still uses this simple pattern.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023 · arxiv.org/abs/2210.03629

ReAct = "Reasoning and Acting", Yao et al. 2022: a model does far better alternating a thought with an action and reading the result than answering in one go. This loop is the picture to remember from today; it comes back in Act 3 with the tests in the observe card, and the three standards are you configuring it: the rulebook it reads before the first thought, the judge it observes, and when the "not yet" arrow stops. Point at the three cards and the two arrows, and then at the stack from three slides ago: same thing, unrolled.

Context.



2026-09-22

Rules, judge, loop

Context.

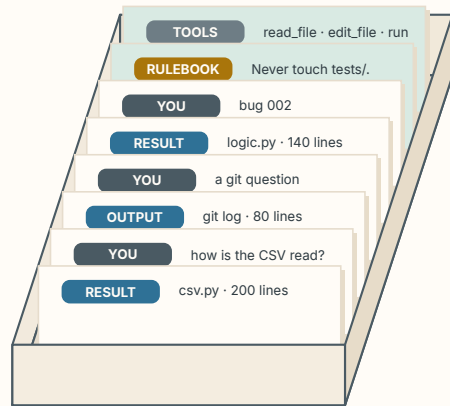
Context.



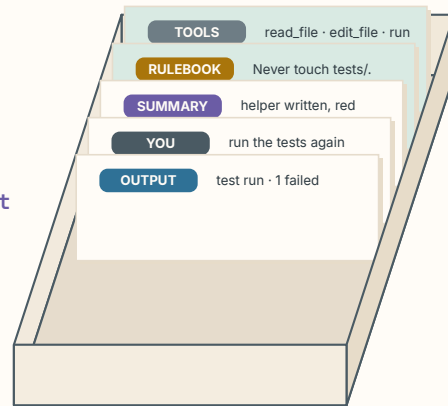
Slow down here; this is the slide that makes "context" concrete. It is one box, and everything in it is a card of text. At the back, put there before the model thinks at all: the tools, a menu of three, one line each saying what it gives back, written by the harness; and the rulebook, which is your AGENTS.md, with two of the lines they add today. In front of them, one card per turn, added as it works: your message, a thought, a tool call, and the result, which is the whole file, now in the box — the newest card is at the front, and the next one goes in front of it. Then the two outlined chips, which answer the question someone asked two slides ago: how does it know which tool? Because the tool's own line is in the box, a few cards behind the thought that reaches for it. It does not "have" tools the way you have hands; it reads a menu and writes an order, and the harness cooks. The model beside the box reads all of it, back to front, before every word it writes, and nothing that is not in the box exists for it. The box fills with every turn; the next slide is what it looks like some way into a bug.

Context rot.

87% FULL



12% FULL



A full, messy box makes the work worse. Always `/compact` it.

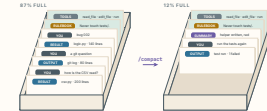
Measured on bug-fixing: Rando et al. (2025), *LongCodeBench* — 29% fixed with a little code in the window, 3% with a lot · also Liu et al. (2024), Chroma (2025).

2026-09-22

Rules, judge, loop

Context rot.

Context rot.



A full, messy box makes the work worse. Always `/compact` it.

Same box, some way into a bug. Left: the rulebook is one thin card at the back now, your own messages a handful, and everything else is files it read and command output — it put them there, not you. And it is not one topic: the bug, then a git question, then how the CSV is read, all still in the box every time it reads it. Type `/context` if you have the screen: the percentage is what it prints. Full and messy is measured, on coding too: Rando et al. 2025 (*LongCodeBench*) gave models real GitHub bugs with more and more of the repository in the window; Claude 3.5 Sonnet fixed 29 percent at 32 thousand tokens and 3 percent at 256 thousand, the same bugs with more text around them. On reading: Liu et al. 2024 found a model did worse with the answer in the middle of twenty documents than with none; Chroma 2025 saw eighteen models fall at copying a list before ten thousand tokens, faster with unrelated text mixed in. Right, after `/compact`: the pile of files became one card, the summary the model wrote of its own conversation; the rulebook and the last turns survive as they were, and the box is mostly empty again. Near the top it compacts on its own; you can ask earlier. One rule follows in the next act: after twice round on the same failure, start a new thread with a better ask.

3

THE CORE OF THE CLASS

Rules, judge, loop.

Instructions tell the agent what to do. Evidence tells you whether it worked.

2026-09-22

Rules, judge, loop

3

THE CORE OF THE CLASS

Rules, judge, loop.

Instructions tell the agent what to do. Evidence tells you whether it worked.

Say the bridge in their words: you are about to hand a task to something that works fast, never gets tired, and will tell you it is done. Point back at the opening slide if it helps: the purple middle of that row is what this act is about. It has three controls: rules, which define the work; a judge, which supplies evidence; and a loop, which uses that evidence to decide what happens next. Every example from here to Your turn is bug 002, the bug they fix in half an hour.

AGENTS.md

AGENTS.md

```
## Running things
uv run streamlit run app.py
uv run pytest

## How to answer
- Every claim about the code comes with file ...
- Change code only when asked to fix one issue ...
- —No git in this project unless the student asks—
+ - Build each spec on a branch named after it, ...
+ - After every change, run the tests for the task ...
+ - Never edit, delete or skip anything in tests/. ...
```

reads it first

$f(x)$

WHAT GOES IN

- commands it cannot guess
- rules you would repeat anyway
- what is off limits

WHAT STAYS OUT

- what it can read in the code

The lines you would otherwise repeat. It reads them before every conversation.

2026-09-22

Rules, judge, loop

AGENTS.md

AGENTS.md



The page is drawn as the diff they will read in a minute: three sections, and the three boundary lines in green, the way GitHub Desktop shows an added line. Same page as the context slide on purpose: the rulebook is the top of that page, read before your message, every new conversation. Ask Codex on stage: "What does AGENTS.md tell you to do?" It recites the file; that is the proof. Then the live part: ask Codex to replace the last line under "How to answer" ("No git in this project unless the student asks", true until last week) with the three lines, read the diff aloud (one red line, three green), accept, commit in GitHub Desktop. Codex only reads AGENTS.md at the start of a thread, so say "new conversation" now. What stays out, if asked: your code explained. It can read that itself, and every extra line hides a real one; the test for a line is "would removing it cause a mistake?" Mention the 32 KB cap only if asked; the real cap is attention.

This task: the specification.

THE TASK TICKET · specs/feature-2-categories.md

WHY

Expenses tagged, so we see where the money went.

WHAT

- ☐ a fixed list of five categories
- ☐ a category column in `expenses.csv`
- ☐ `add_expense` takes a category
- ☐ `category_totals(group_id)`: a total per category

FENCE · OUT OF SCOPE

Editing later · custom categories · charts.

DONE WHEN

Tests green. The seed script runs.
A form box, page totals. **No test: you click.**

WHO USES IT NEXT

ITS REASONING

when a line reads two ways, this is how it picks

THE TESTS

one line, one test: the judge, next slide

THE ASSIGNMENT

the fence goes into the handover card

THE GOAL

the finish line becomes the /goal sentence

2026-09-22

Rules, judge, loop

└ This task: the specification.

This task: the specification.



Last week you read one. In Block 2 you write them: the same four sections.

One minute. The rulebook is the brief for every task; this is the brief for one, and they know the page: it is the spec from last week's slide, feature 2's this time. Walk the four tags, not the lines. Why is the intent: the reasoning slide showed an ambiguous ask, and this is where the model finds which reading is the safe one. What is a checklist, and the co-teacher wrote one test per line, so the checklist is the judge in code. Out of scope is a fence for this task only, the way the rulebook is a fence for all of them. Done when is the finish line in plain words, and point at the red line: one of them has no test, only clicking catches it, which is the green-is-not-done moment later. Land the last caption: in Block 2 there is no PM; they write these, four sections, and the What lines are the tests they ask for first. The full spec is in their repo and on the Week 4 page.

Last week you read one. In Block 2 you write them: the same four sections.

A test is a claim you can run.

```
def test_the_category_list_is_fixed():  
    """The five categories, in this order."""  
    assert logic.CATEGORIES == [  
        "rent", "groceries", "fun",  
        "transport", "other"]
```

→ **its name is the What line**
→ **what it claims, in English**
→ **assert: what must be true**

tests/test_feature_2.py · in your repo now, one of four

true → green false → red

Nobody's opinion. The same answer every time,
before the agent starts and after it stops.

Three lines: a name, the claim, and `assert` — the word that makes it fail. That is the whole idea. Red means the claim is not true yet.

2026-09-22

Rules, judge, loop

└ A test is a claim you can run.

Forty-five seconds, and do not skip it: they have been running tests since Class 3 without ever opening one, and the next slide asks them to check whether each test really tests its What line, which is impossible if a test is a black box that goes red. Read the three parts off the card. The name is the What line in snake case. The docstring is the claim in English. `assert` is the only word that does anything: if what follows is true the test passes silently, and if it is false it stops and prints what it wanted against what it got — which is the failure they will be reading all afternoon. Say the honest limit: a test only checks what someone thought to write down, which is why green is not the end and why one Done when line on the ticket has no test at all. This is the real file in their repo; the other three are the same shape.

A test is a claim you can run.

```
def test_the_category_list_is_fixed():  
    """The five categories, in this order."""  
    assert logic.CATEGORIES == [  
        "rent", "groceries", "fun",  
        "transport", "other"]
```

→ **its name is the What line**
→ **what it claims, in English**
→ **assert: what must be true**

Three lines: a name, the claim, and `assert` — the word that makes it fail. That is the whole idea. Red means the claim is not true yet.

The judge comes from the spec.

1 • THE SPEC • WHAT

☒ a fixed list of five categories

☒ a category column in expenses.csv

☒ add_expense takes a category

☒ category_totals(group_id)

2 • THE TESTS • test_feature_2.py

test_the_category_list_is_fixed

test_expenses_have_a_category_column

test_add_expense_saves_the_category

test_category_totals_for_a_group

3 • ONE VERDICT

```
> pytest  
4 failed  
before you start
```

A JUDGE:

written before the work

not by the agent

same answer every time

These four are in your repo already, and already red. The verdict is theirs, not the agent's. Red is the agent's next input.

2026-09-22

Rules, judge, loop

└─ The judge comes from the spec.

The judge comes from the spec.



Follow the arrows left to right: each What line of the ticket has one test with a name that says the same thing, and the four of them give one verdict. That is the causal backbone of the class: the judge is written from the spec, before the work, by someone who is not the agent, and it gives the same answer every time you run it, which is what the three boxes at the bottom say. Anyone can type "the tests pass", which is why the third rulebook line asks for the output itself: when the sentence and the verdict disagree, the terminal wins. Run the red tests on stage now (command on the Week 4 page) so the real output replaces the box, and open the test file next to the spec so they see the names match. Read one failure bottom up while it is on screen: the last line says how far from done, the assertion message says why and is the to-do written by whoever wrote the test, the name says what is checked. Lines two and three are what the agent is told next; red is evidence, not bad news. In Block 2 they ask for these tests first, one per line, before any code.

The assignment.

HANDED TO THE AGENT · A NEW THREAD, ON THE BRANCH

TASK

FENCE

FINISH LINE

PROOF

Fix **bug 002: removing a member crashes the page**. Nothing else.

Work on **this branch** only. Do not change any test.

Run the tests, and keep going until every one passes.

Then paste the full test output and list every file you touched.

→ $f(x)$
reads it first

The exact object you hand over. Only the two highlighted slots change between tasks.

2026-09-22

Rules, judge, loop

└─ The assignment.

One card, four strips: this is the thing that gets pasted, nothing else. Make them name the four parts back to you before you paste it: the task, the fence, the finish line, the proof. The highlighted slots are the only words that change between tasks; the skeleton is on the Week 4 page. Then say where each strip came from: the fence is the ticket's out-of-scope band plus the rulebook, the finish line is the ticket's Done when, and the proof is what the judge reads. New conversation before pasting: the rulebook lines from five minutes ago are only read at the start of one. Next: what happens when nobody is watching the loop it now runs.

The assignment.

HANDED TO THE AGENT · A NEW THREAD, ON THE BRANCH

TASK

FENCE

FINISH LINE

PROOF

Fix **bug 002: removing a member crashes the page**. Nothing else.

Work on **this branch** only. Do not change any test.

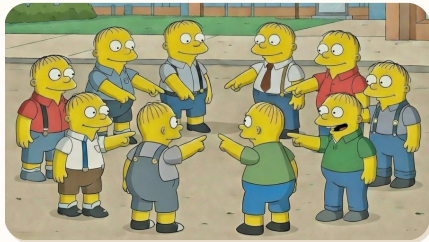
Run the tests, and keep going until every one passes.

Then paste the full test output and list every file you touched.

→ $f(x)$
reads it first

The exact object you hand over. Only the two highlighted slots change between tasks.

The Ralph Wiggum loop.



Cheerfully wrong, and he gets there anyway. A loop around the loop: one run is one attempt, and the same assignment comes back until the evaluation stops it.

The technique, and the name: Huntley (2025), *Ralph Wiggum as a "software engineer"*, ghuntley.com/ralph, a Claude Code plugin since Dec. 2025 · Ralph

Wiggum from *The Simpsons* © 20th Television

2026-09-22

Rules, judge, loop

└─ The Ralph Wiggum loop.

The Ralph Wiggum loop.

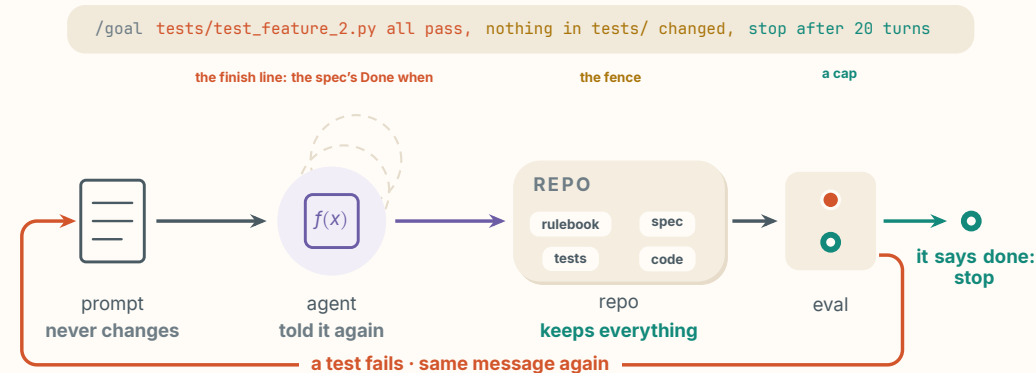


Cheerfully wrong, and he gets there anyway. A loop around the loop: one run is one attempt, and the same assignment comes back until the evaluation stops it.

The technique, and the name: Huntley (2025), *Ralph Wiggum as a "software engineer"*, ghuntley.com/ralph, a Claude Code plugin since Dec. 2025 · Ralph Wiggum from *The Simpsons* © 20th Television

This is a second loop around the agent loop they have just seen, not a smarter agent. Say the distinction explicitly: the inner loop is read, edit, run inside one task. The outer loop returns to the same assignment after a task ends. Both need the same three controls: a rulebook, a judge, and a stop condition. The name is Ralph Wiggum because the process repeats the work cheerfully until external evidence says it can stop. Huntley's original was a shell loop that fed the same prompt file to a fresh agent each run; Anthropic shipped it as a Claude Code plugin in December 2025 ('/ralph-loop "the assignment" -max-iterations 20 -completion-promise GREEN': the assignment, a cap, a phrase it may only say when true, and nothing checks the phrase, so the cap). The next slide draws it, in the form they will actually type today.

Persistent agentic loop.



The message stays fixed. The repo keeps the work. The eval decides whether it runs again.

2026-09-22

Rules, judge, loop

└ Persistent agentic loop.

Persistent agentic loop.



Two minutes. Name the picture first: the Act 2 agent loop again, one level up — the inner loop is read, edit, run inside one task; this outer one returns to the same assignment after a task ends. Walk the row: the agent works in the repo, which holds their rulebook, spec and tests; the eval decides whether the goal is met, and today theirs is the test suite, which is why the judge slide is where it comes from. On red the same assignment goes back in; the ghosts are the runs still to come. The three tags are the point: the message never changes, the agent is told it again, the repo keeps everything. Read the goal sentence aloud — the spec's Done when, plus the fence and a cap — and say it is useful only after the red test exists. Then the trap, which is the judge slide again: Codex stops "when it's confident it has reached the stopping condition", its own word, so the finish line must be something it shows, the test output on screen. (Claude Code's version has a second model read the thread each turn; it cannot run the tests either, which is what the cap is for.) The boundary: a goal persists; it does not create the rulebook, the judge or the human review. Say the setup aloud, as it is no longer on the slide: once, 'codex features enable goals', restart, then /goal plus the sentence in a new thread — both written out on the Week 4 page.

Delegating coding agents.

FIRST, SET IT UP

the first three are already in your repo

- 1 The rules.** Add the three lines, commit. `AGENTS.md`
- 2 The task.** What must be true, what to leave alone. `specs/feature-2-...md`
- 3 The tests.** Run them first: four red, your to-do list. `tests/test_feature_2.py`
- 4 A branch,** named after the task. `feature-2-...`

THEN HAND IT OVER

- 5 One task, in a new chat.** Keep going until the tests pass, then paste the test output.

THEN CHECK IT YOURSELF

- 6 Read the diff. Click the app** — no test covers that. Then commit, push, pull request, merge.

2026-09-22

Rules, judge, loop

└─ Delegating coding agents.

One minute, and it is the slide to photograph. The phases are the lesson: everything in the first group happens before the agent is told anything, the handover is one sentence, and the last group is you, not it. Say what is theirs and what is given — they write the rules today; the spec and the tests are written for them this week, and writing those is what changes in Block 2. Step 5 is the four-part ask in plain words, the same sentence whether they paste it or set it as a goal; the named version is two slides back. File names and the exact ask are on the Week 4 page.

Delegating coding agents.

FIRST, SET IT UP

- 1 The rules.** Add the three lines, commit. `AGENTS.md`
- 2 The task.** What must be true, what to leave alone. `specs/feature-2-...md`
- 3 The tests.** Run them first: four red, your to-do list. `tests/test_feature_2.py`
- 4 A branch,** named after the task. `feature-2-...`

THEN HAND IT OVER

- 5 One task, in a new chat.** Keep going until the tests pass, then paste the test output.

THEN CHECK IT YOURSELF

- 6 Read the diff. Click the app** — no test covers that. Then commit, push, pull request, merge.

Your turn.

- 1 **Branch**, named after the task.
- 2 **Run the tests**. Red is your to-do list.
- 3 **New chat**. Paste the ask, or set `/goal`.
- 4 **Same red twice?** Stop. Read it yourself.
- 5 **Green?** Read the diff. Click the app.
- 6 **Commit, push, pull request, merge**.

SPLITIT

Bug 2, then feature 2

Removing a member crashes the page. Then expense categories.

issues/002 ·
specs/feature-2-categories.md

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then conversion by source.

issues/003 ·
specs/feature-2-conversion.md

2026-09-22

Rules, judge, loop

└─ Your turn.

This slide stays on screen for forty minutes. First the live ten: do feature 2 on stage and let it loop for real, 2–4 rounds is normal. Narrate the workflow: "that's a read, that's an edit, that's a run — red — round again." Commands and file names are on the Week 4 page if you want them on screen. The two things it commonly misses on this feature: it forgets to refresh the data files from the seed (tests stay green while the app crashes on load) and it never adds the form field (nothing tests that screen). Both are the "green is not done" lesson. Show them, do not fix them silently. If it edits a test: stop, discard, point at the rule, re-prompt. Then their thirty: roam for two things, did they ask for the test output, and did they read the diff before accepting. Never answer a failing test; ask "what does the last line say?". SplitIt bug 002: the test demands a helper that also removes the member's expenses; the assertion message says so. Tiny CRM bug 003 has a twist: stamping the close date is not enough. The month counter gives up on the whole list when any won lead has no date, and the seed has five; the fix is to skip them, not return zero. Anyone still on feature 1 does that first; feature 2 becomes homework.

Your turn.

- 1 Branch, named after the task.
- 2 Run the tests. Red is your to-do list.
- 3 New chat. Paste the ask, or set `/goal`.
- 4 Same red twice? Stop. Read it yourself.
- 5 Green? Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

SPLITIT

Bug 2, then feature 2
Removing a member crashes the page. Then expense categories.
issues/002 ·
specs/feature-2-categories.md

TINY CRM

Bug 3, then feature 2
"Won this month" is always zero. Then conversion by source.
issues/003 ·
specs/feature-2-conversion.md

When it goes wrong.

- 1 **It edited a test.** Discard. Say the rule. Run again.
- 2 **"Done" — tests red.** Ask for the output.
- 3 **Same failure, twice.** Stop. Read it yourself. New thread.
- 4 **It fixed three other things.** Discard. One task per loop.
- 5 **Green, app wrong.** Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

2026-09-22

Rules, judge, loop

└─When it goes wrong.

When it goes wrong.

- 1 **It edited a test.** Discard. Say the rule. Run again.
- 2 **"Done" — tests red.** Ask for the output.
- 3 **Same failure, twice.** Stop. Read it yourself. New thread.
- 4 **It fixed three other things.** Discard. One task per loop.
- 5 **Green, app wrong.** Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

2026-09-22

Rules, judge, loop

└─ Write one rule.

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

Collect the rules on the board; the good ones become the course's AGENTS.md for Block 2. Expect: "refresh the data after changing the seed files", "don't edit live data by hand", "show me the diff before committing". Reject vague ones ("be careful"). Ask "how would you know it followed this?".

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rules · judge · loop.

2026-09-22

Rules, judge, loop

└ Before next week.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rules · judge · loop.

Nothing to hand in. Next week's check-in: feature 2 merged, and a commit on AGENTS.md. Part II of the standards — walking away: scripted loops, goals, a second agent reviewing — comes with the team project in Block 3.